

字符串-板子+笔记

·数据类型转换

```
//字符串转整型：
1. int x = s[i] - '0';
2. int num = stoi(s);
2. ll num = stoll(s);
//·字符串转浮点型：
3. atof函数：将string类型转换成double类型的函数
```

·小结-三个函数 (stoi、stod、atof)

·共同特性：

2.遇到非数字,截取停止,即使后面有数字也不会继续读取了

·atof函数的个性：

1.未找到时返回值为0

2.stoi/stod函数使用对象为string类型,而atof函数为const char*

·stoi/stod函数适应性较好,可以读取string类型的字符串

·字符串切片

·使用 substr() 方法 (最常用)

```
string s = "Hello world";
string slice1 = s.substr(6); // 从位置 6 开始，取到字符串末尾
cout << slice1 << endl; // 输出：world
string slice2 = s.substr(0, 5); // 从位置 0 开始，取 5 个字符
cout << slice2 << endl; // 输出：Hello
```

·使用构造函数

```
// 从位置 3 开始，取 4 个字符
string slice(s, 3, 4);
cout << slice << endl; // 输出：lo w
```

·使用迭代器

```
// 从第2个字符到第7个字符（不包括第7个）
string slice(s.begin() + 1, s.begin() + 7);
cout << slice << endl; // 输出：ello w
```

·拓1：字符串切片赋值（替换部分字符串）

```
// 将第6个字符开始的5个字符替换为 "There"
s.replace(6, 5, "There");
cout << s << endl; // 输出：Hello There
```

·拓2：字符串切片删除

·使用 erase() 方法（最常用）

1.**删除单个字符：**

```
string s = "Hello world";  
s.erase(5, 1); // 删除第5个位置的字符（索引从0开始）  
cout << s << endl; // 输出: He1l world
```

2.**删除一段字符：**

```
s.erase(5, 6); // 从位置5开始，删除6个字符  
cout << s << endl; // 输出: Hello
```

3.**删除从指定位置到末尾：**

```
s.erase(5); // 从位置5开始删除到末尾  
cout << s << endl; // 输出: Hello
```

·使用迭代器删除

1.**删除单个字符：**

```
s.erase(s.begin() + 3); // 删除单个字符（第3个）  
cout << s << endl; // 输出: He1d
```

2.**删除一段字符：**

```
s.erase(s.begin() + 2, s.begin() + 7); // 删除第2到第7个字符（不包括第7个）  
cout << s << endl; // 输出: He1d
```

·删除特定字符或模式

```
string s = "abc123def456ghi";
```

1. 删除所有数字

方法一：for循环

```
for (size_t i = 0; i < s.length(); ) {  
    if (isdigit(s[i])) s.erase(i, 1);  
    else i++;  
}  
cout << s << endl; // 输出: abcdefghi
```

方法二：更高效的方法（使用remove算法）

```
string s2 = "abc123def456ghi";  
s2.erase(remove_if(s2.begin(), s2.end(), ::isdigit), s2.end());  
cout << s2 << endl; // 输出: abcdefghi
```

3.**删除开头和结尾的空白字符**

```
string s = "  Hello world  "; // 删除开头空白
s.erase(0, s.find_first_not_of(" \t\n\r")); // 删除结尾空白
s.erase(s.find_last_not_of(" \t\n\r") + 1);
cout << "[" << s << "]" << endl; // 输出: [Hello world]
```

·字符串大小比较

1.**拼接字符串大小比较**

```
void work(){
    int n;
    cin>>n;
    string s="";
    vector<string> a(n);
    for(int i=0;i<n;i++){
        cin>>a[i];
        // if(i==0) s+=a[i];
        // else if(a[i]<s) s=a[i]+s;
        // else if(a[i]>=s) s+=a[i];
        string s1=s+a[i];
        string s2=a[i]+s;
        if(s1<s2) s=s1;
        else s=s2;
    }
    cout<<s<<endl;
}
```

·一些函数

·字符分类函数

```
isalpha(c); // 是否是字母 (a-z, A-Z)
isdigit(c); // 是否是数字 (0-9)
islower(c); // 是否是小写字母 (a-z)
isupper(c); // 是否是大写字母 (A-Z)
isspace(c); // 是否是空白字符 (空格、制表符、换行等)
isalnum(c); // 是否是字母或数字
```

·字符转换函数

```
tolower(c); // 转换为小写字母
toupper(c); // 转换为大写字母
```

·字符串查询函数

`strstr(s1,s2);` // 作用: 判断s2是否为s1的子串。

如果没找到, 返回NULL;

如果找到了, 返回这个子串第一个字符的地址。

eg: `char s1[]="habch", s2[]="abc"`

`strstr(s1,s2)` 就返回s1中'a'的地址

·cin 之后 `getline()`

```
cin.ignore(numeric_limits<streamsize>::max(), '\n'); // 高配版
```

```
string tmp; // 把cin之后的废料全部当成一整个字符串 （耗时间耗内存）  
getline(cin, tmp); // 低配版
```